



UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Sistemas.

Estructura de Datos

Prof. G. A. Salinas

Semana 14

Grafos Algoritmos

Ciclo 2025-2

Agenda

1. Recorridos
2. Usos de recorridos
3. Orden topológico
4. Grafos con pesos
5. Grafos algoritmos
6. Algoritmo Dijkstra
7. Algoritmo Floyd
8. Algoritmo Washall
9. Albol de cubrimiento
10. Circuitos camino de Euler
11. Ciclo de Hamilton
12. Resumen
13. Referencias

1. Recorridos

- **Recorrido.** Es un procedimiento sistemático de exploración de un grafo mediante el examen de todos sus vértices y aristas.
- Un recorrido es eficiente si visita todos los vértices y aristas en un tiempo proporcional a su número, o sea en tiempo lineal.
- Tipos:
 - Recorrido en profundidad
 - Recorrido en anchura
- Usos:
 - Grafos acíclicos
 - Orden topológico.

2. Uso de los Recorridos

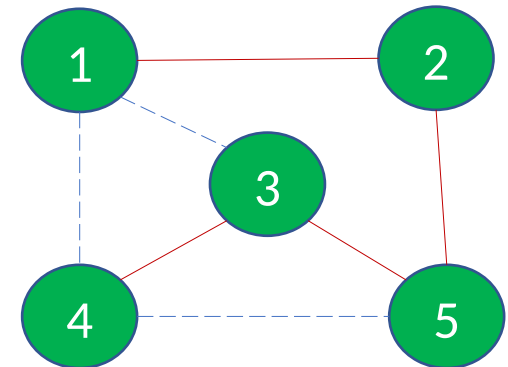
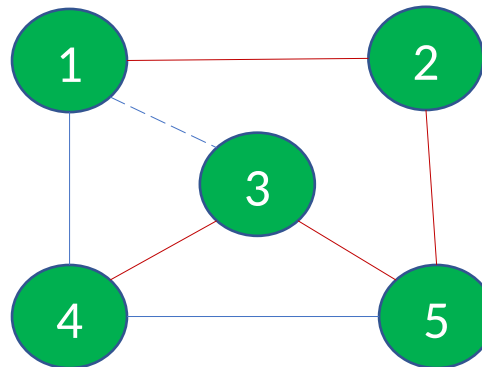
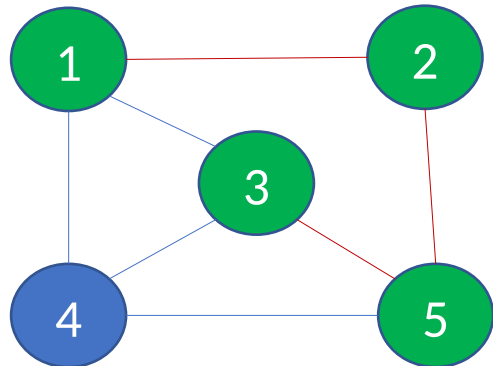
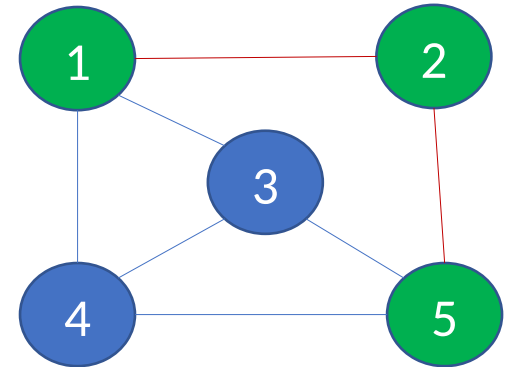
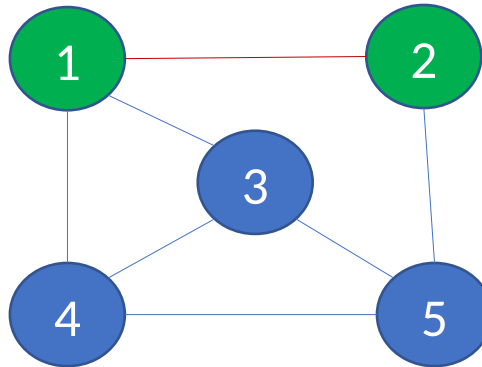
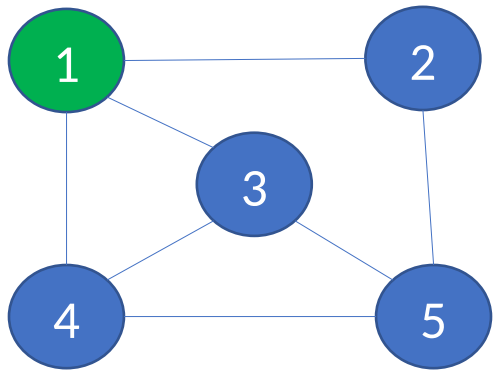
- Los recorridos en Profundidad y Anchura se pueden utilizar para resolver los siguientes problemas:
 - Analizar que el grafo es conectado.
 - Obtener un árbol de expansión del grafo.
 - Obtener componentes conectados al grafo.
 - Obtener un camino entre dos vertices dados del grafo o que no existe.
- El recorrido en profundidad se utiliza:
 - Obtener un ciclo del grafo o que no tiene ciclos.
- El recorrido en anchura se utiliza para:
 - Obtener para cada vertice v del grafo, el número mínimo de aristas de cualquier camino entre w y v

2. Recorrido en profundidad

- Es una generalización del recorrido preorden de un árbol.
- Se pueden identificar aristas de cuatro tipos en el recorrido:
 - **Aristas de descubrimiento.** Conducen al descubrimiento de nuevos vertices, denominados tambien aristas del árbol.
 - **Aristas de retorno.** Conducen a vertices antecesores ya visitados.
 - **Aristas de avance.** Conducen a vertices descendientes en el árbol.
 - **Aristas de cruce.** Conducen a un vertices que no es ancestro de otro o a vertices que estan en diferentes arboles
- Las aristas de descubrimiento forman un árbol de cubrimiento de los componentes del vertice inicial.

2. Recorrido en profundidad

- Es una generalización del recorrido preorden de un arbol.

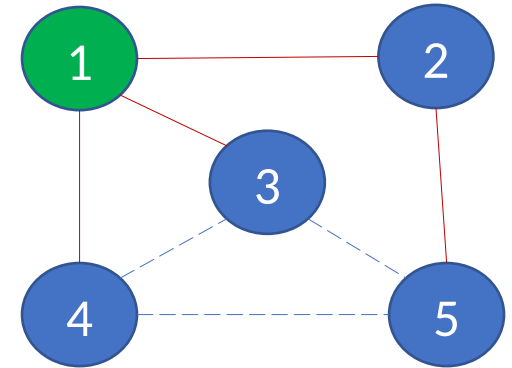
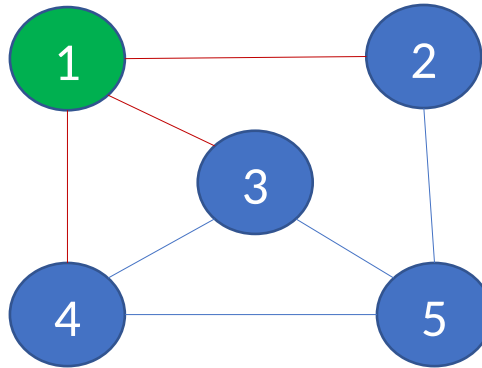
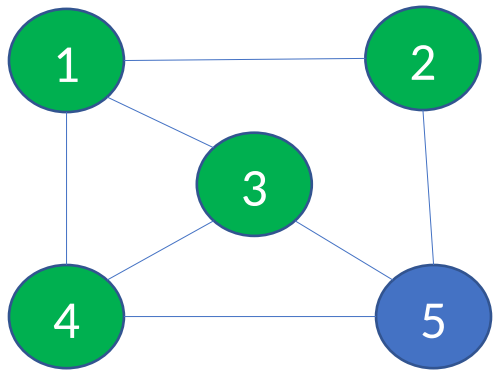


2. Recorrido en amplitud

- Es una generalización del recorrido por niveles de un árbol.
- Se pueden identificar aristas de dos tipos en el recorrido:
 - **Aristas de descubrimiento.** Conducen al descubrimiento de nuevos vértices.
 - **Aristas de cruce.** Conducen a un vértice ya visitado.
- Las aristas de descubrimiento forman un árbol de cubrimiento de los componentes del vértice inicial.

2. Recorrido en ~~profundidad~~ amplitud

- Es una generalization del recorrido por niveles de un arbol.



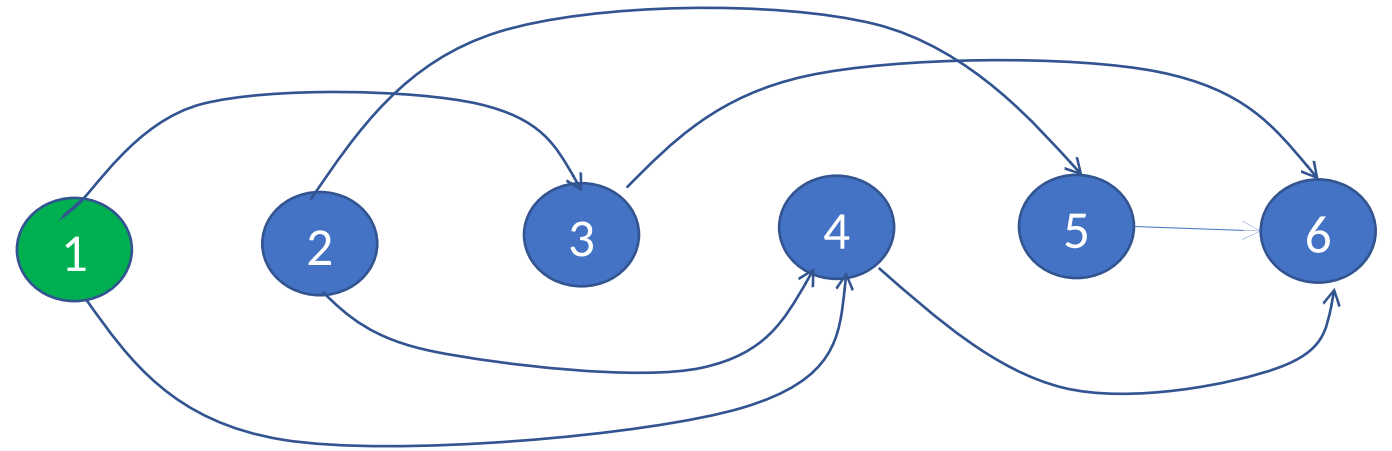
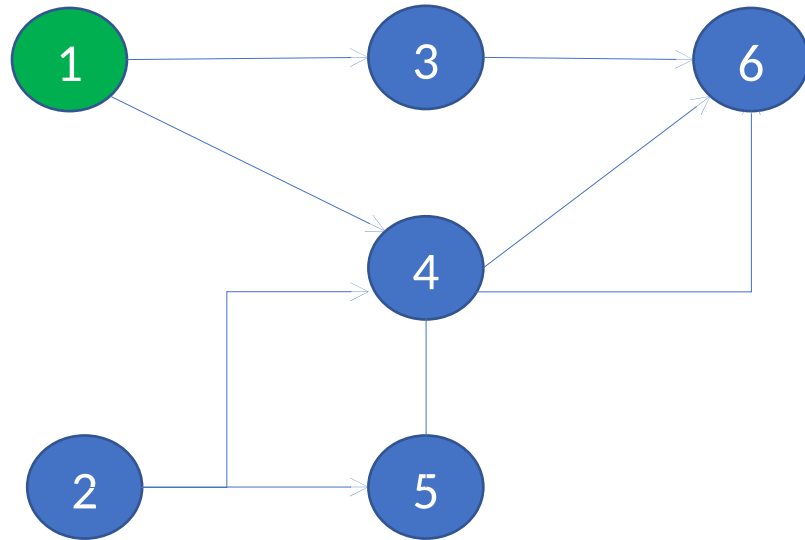
3. Orden topologico

- **Ordenamiento topologico de un grafo aciclico.** Orden lineal de los vertices de tal manera que todas las aristas tengan una direccion de izquierda a derecha. Ej. Los cursos del plan de estudios
- El algoritmo es una version modificada del recorrido en anchura.

```
ACCION ordenTopologico (v) /* orden inverso */
    marca[v]=VISITADO
    PARA cada vertice w en listaAdyacencia (v)
        SI(marca[w] = SINVISITAR)
            ordenTopologico(w);
        FIN_SI
    FIN_PARA
    imprime(v);
FIN_ACCION
```

3. Orden topológico

- Ejemplo:



Orden Topologico:

1 2 3 4 5 6

1 3 2 4 5 6

2 1 5 3 4 6

4. Grafos con pesos

- Cada arista lleva asociado un valor numerico no negativo $w(p)$, que representa el costo que varia linealmente a lo largo del camino que puede ser distancia, tiempo, etc.
- El costo de un camino es la suma de los costos de las aristas en el camino.
 - El grafo que representa las rutas entre ciudades de una region, pais. El peso de las aristas representa el tiempo o la distancia entre ciudades

5. Grafos, algoritmos

- El problema de los caminos mas cortos desde el vertice:.
 - Algoritmo de Dijkstra.
- El problema de los caminos mas cortos entre los pares de vertices:
 - Algoritmo de Floyd.
- Cierre transitivo: Es la relacion binaria mas pequena.
 - Algoritmo de Warshall
- Arbol de cubrimiento de costo minimo.
 - Algoritmo de Prim.
 - Algoritmo Kruskal.

6. Algoritmo de Dijkstra

La idea principal es realizar una búsqueda a lo ancho “ponderada” empezando por el vértice inicial f .

- De manera iterativa se construye un conjunto de vértices seleccionados que se toman del conjunto de vértices candidatos C según el menor peso (distancia) desde f .
- El algoritmo termina cuando no hay más vértices de G fuera del conjunto formado.
- El paradigma usado corresponde al **método voraz**, en el que se trata de optimizar una función sobre una colección de objetos (menor peso).
- Se usa un vector $d[v]$ para almacenar la distancia de v a f .
- Complejidad:
 - Con matrices de adyacencia: $O(n^2)$
 - Con listas de adyacencia: $O(n^2)$
 - Utilizando una cola de prioridad para buscar el vertice con menor $d[v]$ se consigue $O(n \log n)$

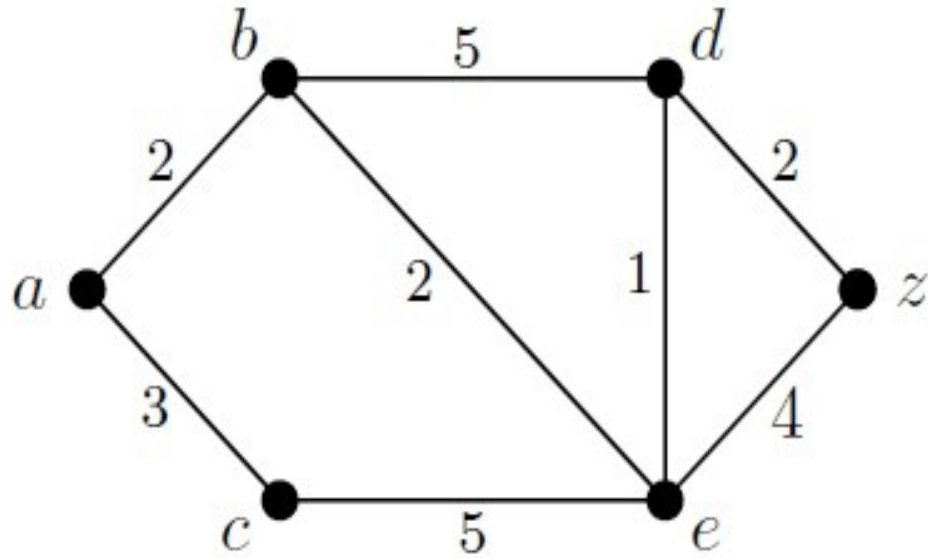
6. Algoritmo de Dijkstra

Caminos mínimos y distancias [Garcia 2015]

- Camino mínimo y distancia entre dos vértices (Dijkstra). Aplicaciones: navegadores, ubicación de servicios, etc.
 - **Camino mínimo:** camino de menor peso que une ambos vértices (¿único?)
 - **Distancia:** es el peso del camino mínimo
 - **Estrategia:** partir de un vértice e ir calculando el camino mínimo y la distancia al resto de los vértices, en orden creciente de distancia

6. Algoritmo de Dijkstra

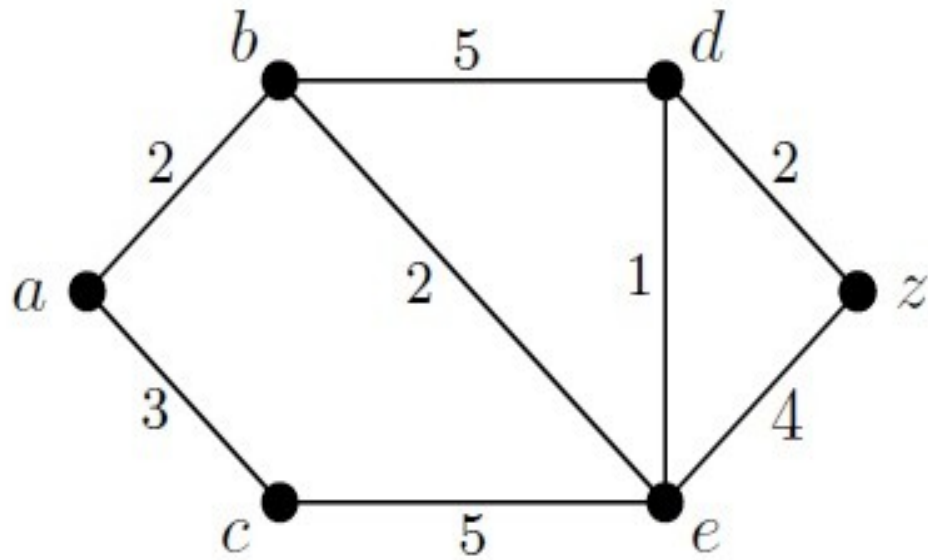
Caminos mínimos y distancias [Garcia 2015]



Caminos	a	b	c	d	e	z
	0					

6. Algoritmo de Dijkstra

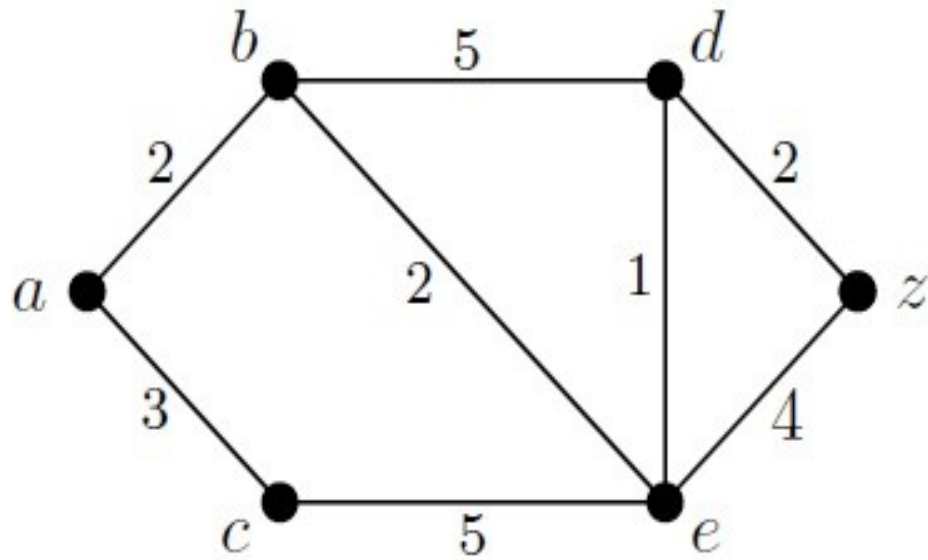
Caminos mínimos y distancias [Garcia 2015]



Caminos	a	b	c	d	e	z
A, b	0	2	3			

6. Algoritmo de Dijkstra

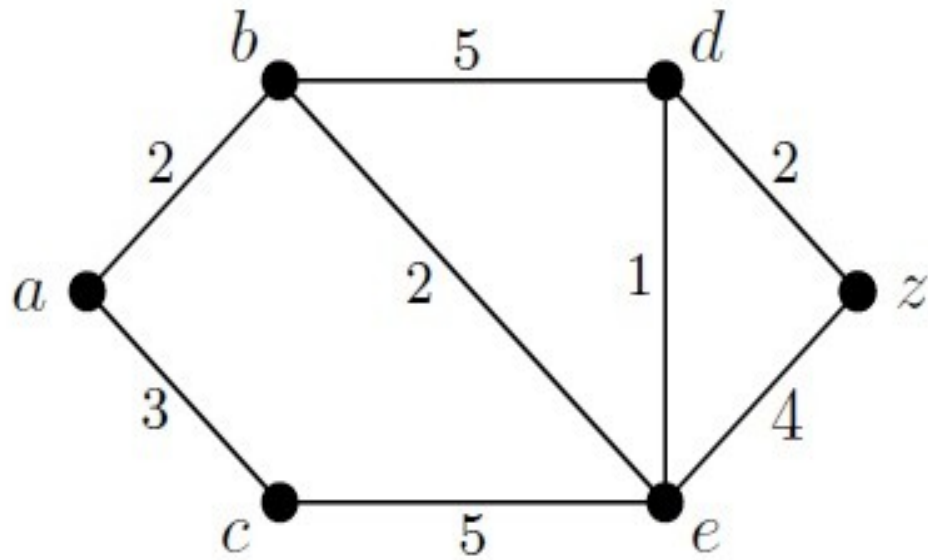
Caminos mínimos y distancias [Garcia 2015]



Caminos	a	b	c	d	e	z
A, b	0	2	3			
A, c			3	7	4	

6. Algoritmo de Dijkstra

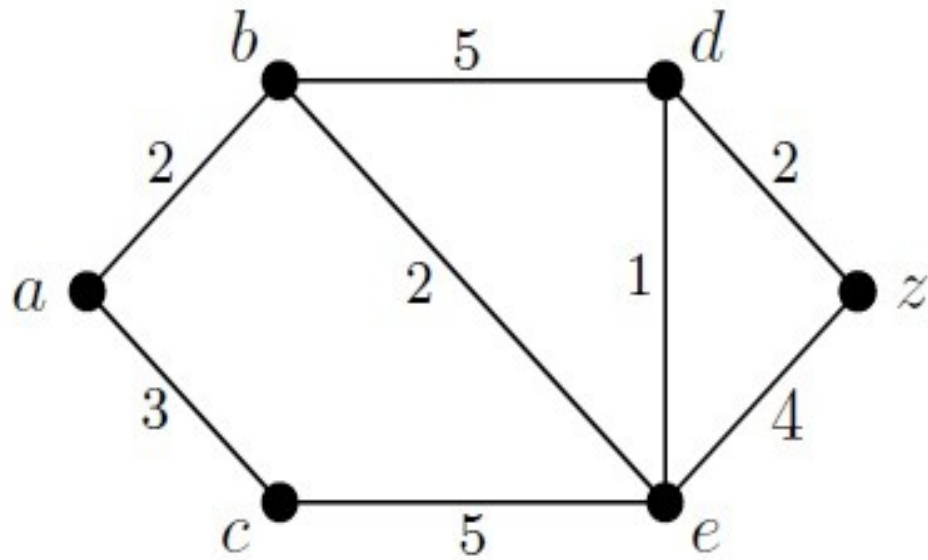
Caminos mínimos y distancias [Garcia 2015]



Caminos	a	b	c	d	e	z
A, b	0	2	3			
A, c			3	7	4	
A, b, e				7	4	
A, b, e, d,				5		8

6. Algoritmo de Dijkstra

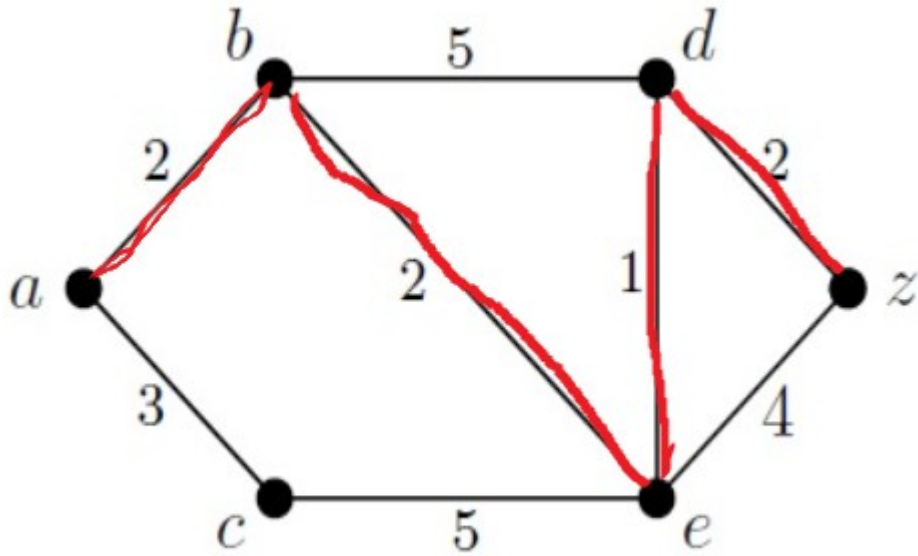
Caminos mínimos y distancias [Garcia 2015]



Caminos	a	b	c	d	e	z
A, b	0	2	3			
A, c			3	7	4	
A, b, e				7	4	
A, b, e, d,				5		8
A, b, e, d, z						7

6. Algoritmo de Dijkstra

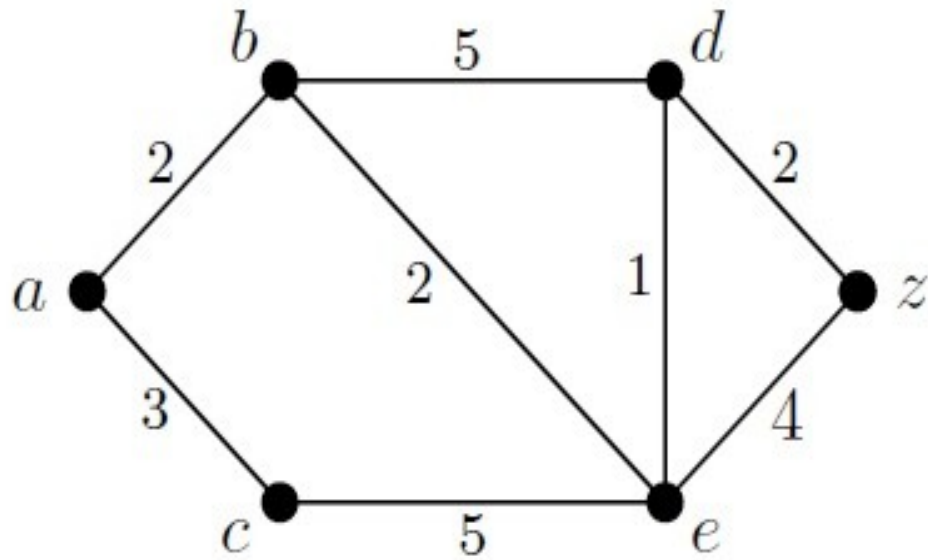
Caminos mínimos y distancias [Garcia 2015]



Caminos	a	b	c	d	e	z
A, b	0	2	3			
A, c			3	7	4	
A, b, e				7	4	
A, b, e, d,				5		8
A, b, e, d, z						7

6. Algoritmo de Dijkstra

Caminos mínimos y distancias. Tabla de distancias [Garcia 2015]

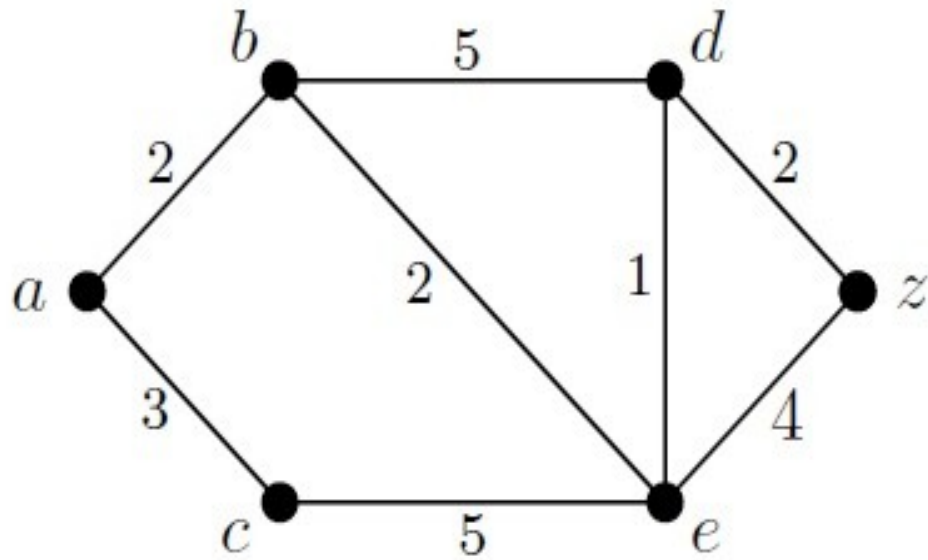


Caminos			a	b	c	d	e	z
A			0	2	3	5	4	7
B				0				
C					0			
D						0		
E							0	
Z								0

Es simetrica, en la diagonal hay ceros

6. Algoritmo de Dijkstra

Caminos mínimos y distancias. Tabla de distancias [Garcia 2015]



Caminos	R()	S()	a	b	c	d	e	z
A	7	21	0	2	3	5	4	7
B	5	17	2	0	5	3	2	5
C	8	27	3	5	0	6	5	8
D	6	17	5	3	6	0	1	6
E	5	15	4	2	5	1	0	5
Z	8	25	7	5	8	2	3	0

Es simetrica, en la diagonal hay ceros.

Centros del Grafo. Son los vertices que minimizan la maxima distancia.

7. Algoritmo de Floyd

Algoritmo de Floyd-Warshall.

Algoritmo de análisis de grafos para determinar el camino mínimo entre nodos en grafos ponderados.

- **Algoritmo de Warshall.** Es un algoritmo booleano que inicia con CEROS no ha correspondencia inicial en el grafo y de UNOS hay correspondencia (fleche, caminos entre nodos), obtiene una matriz denominada Matriz de clausura transitive en la que se muestra todas las posibles unions entre nodos (directa o indirectamente)
- Algoritmo de Floyd. Es similar pero lo hace el grafos con pesos, es decir el valor de la fleche o caminp es un valor o ∞ INF (no hay fecha o camino) que representa la matriz

https://es.wikipedia.org/wiki/Algoritmo_de_Floyd-Warshall

9. Arbol de cubrimiento de costo minimo

Un *árbol de cubrimiento o expansión* para un grafo $G = (V, E)$ no dirigido conectado con pesos es un árbol libre que conecta todos los vértices en V .

- El costo de un árbol de cubrimiento está determinado por la suma de los costos de las aristas en el árbol.

Problema: hallar el árbol de cubrimiento de costo mínimo para G .

- Problema común en el planeamiento de redes de distribución y comunicación.
- Se soluciona con los algoritmos de Prim o Kruskal

Complejidad:

- Ambos algoritmos son de tipo voraz: se selecciona uno de los candidatos con el criterio que es mejor en cada momento (menor costo)
- Con matrices de adyacencia: $O(n^2)$
- Para grafos dispersos es mejor usar la versión Dijkstra con lista de adyacencia que toma $O(n \log n)$

10. Camino y circuitos de Euler

Aplicación. Un grafo no dirigido se utiliza para representar un dibujo de líneas

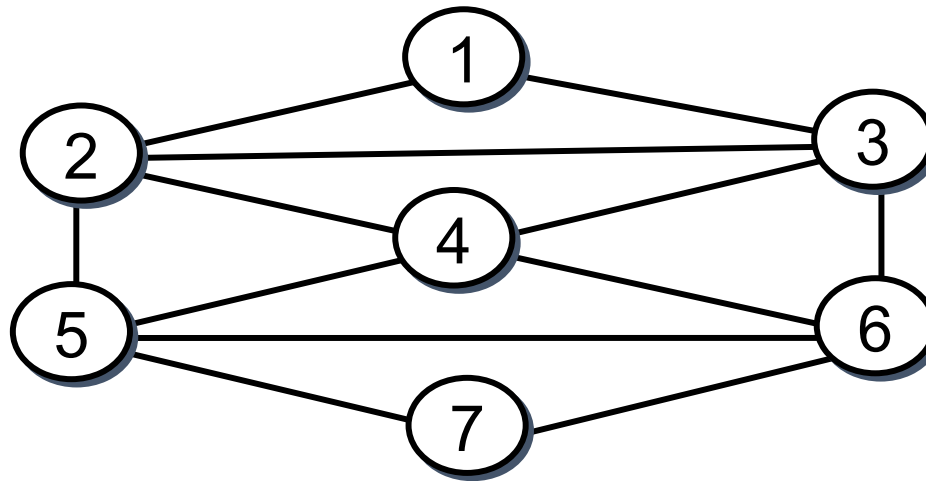
- **Circuitos de Euler.** Es un ciclo que visita todas las aristas exactamente una vez.
- **Camino de Euler.** Se puede empezar y acabar en nodos distintos.

Condiciones necesarias:

- El grafo debe ser conexo.
- Todos los nodos deben tener grado par, ya que el camino entra y sale de los nodos.

10. Algoritmo de Euler

- **Algoritmo de un circuito de Euler.** Un grafo no dirigido se utiliza para representar un dibujo de líneas
 1. Buscar un ciclo cualquiera en el grafo empezando por v.
 2. Si quedan aristas por visitar, seleccionar el primer nodo, w, del ciclo que tenga una arista sin visitar. Buscar otro ciclo partiendo de w que pase por las aristas no visitadas.
 3. Unir el ciclo del paso 1 con el obtenido en el paso 2.
 4. Repetir sucesivamente los pasos 2 y 3 hasta que no queden aristas por visitar.



11. Ciclo de Hamilton

Ciclo hamiltoniano. Aplica a grafos no dirigidos.

- **Definición..** Es un ciclo SIMPLE que visita todas los vértices, es decir pasa por todos los vértices exactamente una vez.
- **Problema.** Determinar si un grafo no dirigido tiene un ciclo hamiltoniano o no.

Resumen

- Podemos utilizar grafos para modelar de la realidad.
- Importancia del estudio de problemas genéricos sobre grafos.
- La búsqueda en profundidad es una herramienta básica en mucho de los problemas estudiados.
- Problema de agente viajero.
- Problema de coloración de grafos. Los grafos presentan relaciones de incompatibilidad.
 - Ej. ¿De cuantos colores como mínimo se puede pintar un mapa?. Dos regiones adyacentes no pueden tener el mismo color.
- Comparación e isomorfismo de grafos.

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Villalobos J. (1996) Diseño y Manejo de Estructuras de Datos en C. Santa Fe de Bogota, Colombia: McGraw-Hill.
- Ficheros en C y C++ .[Ultima actualización 13.06.2017] [Accesado en Lima, 19.05.2020, 1820 horas], <http://c.conclase.net/ficheros/>
- García J. (2015) Algoritmos en Teoría de grafos. Escuela Técnica Superior de Ingeniería de Sistemas Informáticos. Universidad Politécnica de Madrid. [Accesado en Lima, 2130 horas de 14 de Enero de 2021]